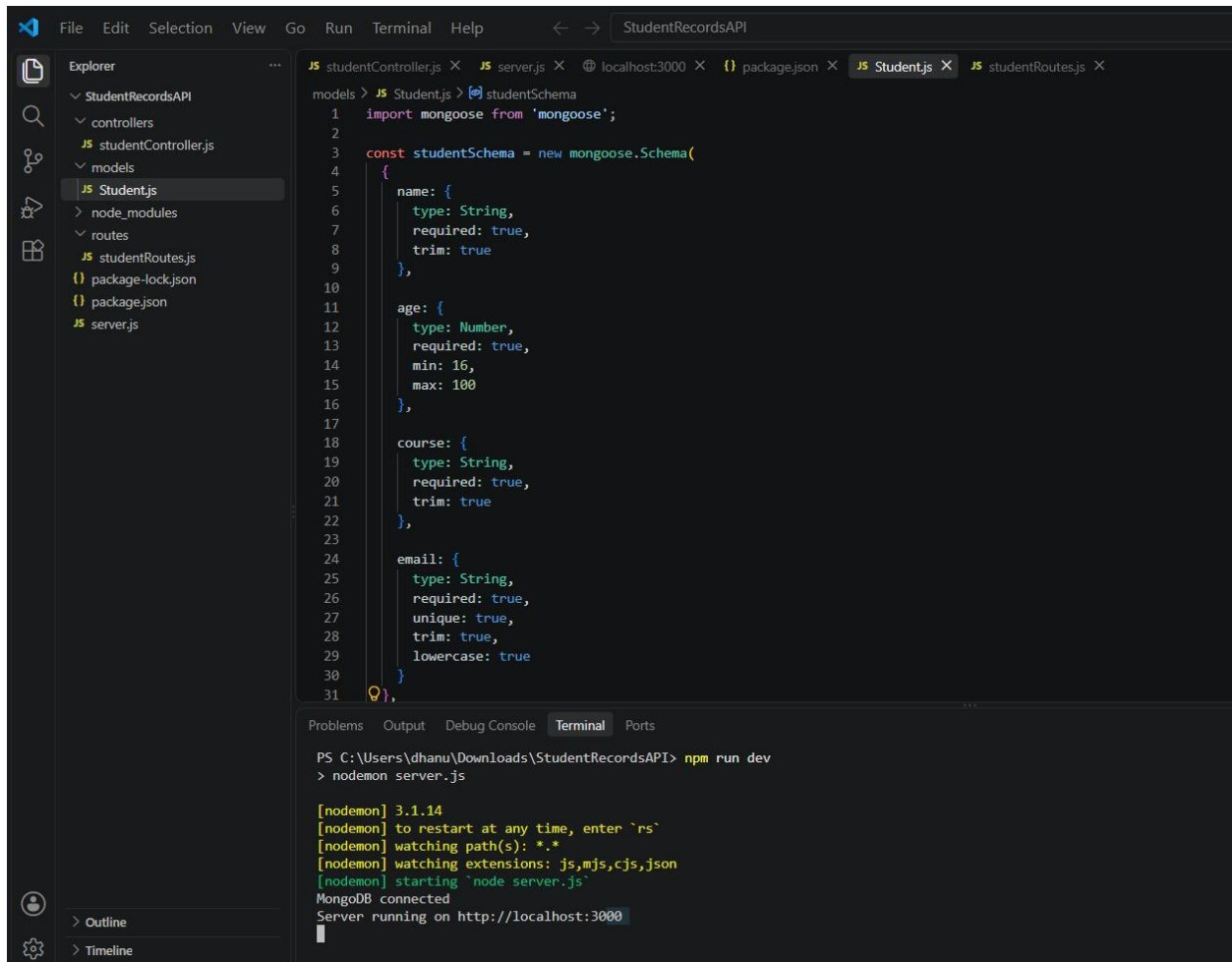


POSTMAN STUDENT RECORDS CRUD API PROJECT

NAME: P.Dhana sri

Roll no:2520030115

VS CODE:



The screenshot shows the VS Code editor interface for a project named 'StudentRecordsAPI'. The Explorer sidebar on the left displays the project structure, including folders for 'controllers', 'models', 'node_modules', and 'routes', and files for 'studentController.js', 'studentRoutes.js', 'package-lock.json', 'package.json', and 'server.js'. The 'models' folder is expanded, showing the 'studentSchema.js' file. The main editor area displays the content of 'studentSchema.js', which defines a Mongoose schema for a student. The schema includes fields for 'name', 'age', 'course', and 'email', each with specific validation rules. The 'name' field is a required string with trimming. The 'age' field is a required number between 16 and 100. The 'course' field is a required string with trimming. The 'email' field is a required string, unique, with trimming and lowercase conversion. The 'Terminal' panel at the bottom shows the command prompt running 'npm run dev' and 'nodemon server.js', indicating the application is running on http://localhost:3000.

```
1 import mongoose from 'mongoose';
2
3 const studentSchema = new mongoose.Schema(
4   {
5     name: {
6       type: String,
7       required: true,
8       trim: true
9     },
10
11     age: {
12       type: Number,
13       required: true,
14       min: 16,
15       max: 100
16     },
17
18     course: {
19       type: String,
20       required: true,
21       trim: true
22     },
23
24     email: {
25       type: String,
26       required: true,
27       unique: true,
28       trim: true,
29       lowercase: true
30     }
31   },
32   { timestamps: true }
33 );
```

PS C:\Users\dhana\Downloads\StudentRecordsAPI> npm run dev
> nodemon server.js

[nodemon] 3.1.14
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node server.js`
MongoDB connected
Server running on http://localhost:3000

GET:

The screenshot shows a REST client interface with a single tab for a GET request to `http://localhost:3000/students`. The interface includes a 'Send' button and tabs for 'Params', 'Auth', 'Headers (6)', 'Body', 'Pre-req.', 'Tests', and 'Settings'. The 'Body' tab is active, showing a 'raw' view of the response in JSON format. The response status is '200 OK' with a response time of '66 ms' and a size of '425 B'. The JSON response is displayed in a 'Pretty' view, showing a single student object with fields for _id, name, age, course, email, createdAt, updatedAt, and __v.

Import GET http://localhost:3000/stu + ...

HTTP http://localhost:3000/students Save

GET http://localhost:3000/students Send

Params Auth Headers (6) Body Pre-req. Tests Settings Cookies Beautify

raw JSON

1

Body 200 OK 66 ms 425 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   {
3     "_id": "6aaa608e41ffd1d730d39cc1",
4     "name": "Anita",
5     "age": 20,
6     "course": "CSE",
7     "email": "anita@example.com",
8     "createdAt": "2026-09-16T09:25:34.704Z",
9     "updatedAt": "2026-09-16T09:25:34.704Z",
10    "__v": 0
11  }
```

POST:

Home Workspaces Explore Search Postman Sign in Create Account

You are using the Lightweight API Client, sign in or create an account to work with collections, environments and unlock all free features in Postman.

History New Import POST http://localhost:3000/students

Today POST http://localhost:3000/students

POST http://localhost:3000/students

Params Authorization Headers (8) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary JSON

```
1 {
2   "name": "Anita",
3   "age": 20,
4   "course": "CSE",
5   "email": "anita@example.com"
6 }
```

Body Cookies Headers (7) Test Results Status: 201 Created Time: 120 ms Size: 481 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "message": "Student created successfully",
3   "student": {
4     "name": "Anita",
5     "age": 20,
6     "course": "CSE",
7     "email": "anita@example.com",
8     "_id": "6aaa608e41ffd1d730d39cc1",
9     "createdAt": "2026-09-16T09:25:34.704Z",
10    "updatedAt": "2026-09-16T09:25:34.704Z",
11    "__v": 0
  }
}
```

Console Not connected to a Postman account

PATCH:

New Import GET http://localhost:3000/stu...

aaa608e41ffd1d7... http://localhost:3000/students/6aaa608e41ffd1d730d39cc1

GET http://localhost:3000/students/6aaa608e41ffd1d730d39cc1

Params Authorization Headers (8) Body Pre-request Script Tests Settings

none form-data x-www-form-urlencoded raw binary JSON

```
1 {
2   "age": 21,
3   "course": "AI & DS"
4 }
```

Body Cookies Headers (7) Test Results Status: 200 OK Time: 41 ms Size: 423 B Save Response

Pretty Raw Preview Visualize JSON

```
1 {
2   "_id": "6aaa608e41ffd1d730d39cc1",
3   "name": "Anita",
4   "age": 20,
5   "course": "CSE",
6   "email": "anita@example.com",
7   "createdAt": "2026-09-16T09:25:34.704Z",
8   "updatedAt": "2026-09-16T09:25:34.704Z",
9   "__v": 0
10 }
```

DELETE:

DEL http://localhost:3000/stuc + ...



http://localhost:3000/students/6aaa608e41ffd1d730d39cc1

Save

DELETE

http://localhost:3000/students/6aaa608e41ffd1d730d39cc1

Send

Params Auth Headers (6) Body Pre-req. Tests Settings

Cookies

raw

JSON

Beautify

1

Body



200 OK

16 ms

476 B

Save Response

Pretty

Raw

Preview

Visualize

JSON



```
1
2   "message": "Student deleted successfully",
3   "student": {
4     "_id": "6aaa608e41ffd1d730d39cc1",
5     "name": "Anita",
6     "age": 20,
7     "course": "CSE",
8     "email": "anita@example.com",
9     "createdAt": "2026-09-16T09:25:34.704Z",
10    "updatedAt": "2026-09-16T09:25:34.704Z",
11    "__v": 0
```

GET http://localhost:3000/ x + ...

HTTP

http://localhost:3000/students/6aaa608e41ffd1d730d39cc139cc1

Save

GET

http://localhost:3000/students/6aaa608e41ffd1d730d39cc1

Send

Params

Auth

Headers (6)

Body

Pre-req.

Tests

Settings

Cookies

Beautiful

raw

JSON

1

Body

404 Not Found 13 ms 273 B Save Response

Pretty

Raw

Preview

Visualize

JSON

1

2

3

{

"message": "Student not found"

}